

Software Requirements Specification

for

Arena: A Real-Time Multiplayer Combat Game

Version 1.0 — PRELIMINARY DRAFT

Prepared by:

Marshall Williams — Server Architecture, Real-Time Systems, Testing Infrastructure, Deployment

Raj Kumar Partel — Client Application & Real-Time UX

En Zhang — Game Design, Data Layer, and Persistence

Software Engineering Term Project

July 2026

NOTE: This is a preliminary draft, prepared to establish a documented, verifiable requirements baseline for the Arena system to guide its development. It is intended as a working input to a final revision pass against course-specific standards.

Table of Contents

Table of Contents	2
Authorship — Primary Responsibility by Section	4
1. Introduction	5
1.1 Purpose	5
1.2 Problem Statement.....	5
1.3 Scope.....	5
1.4 Definitions, Acronyms, and Abbreviations	5
1.5 References	6
1.6 Overview	6
2. Overall Description.....	8
2.1 Product Perspective	8
2.2 Product Functions	8
2.3 User Characteristics	8
2.4 Operating Environment	8
2.5 Constraints.....	9
2.6 Assumptions and Dependencies.....	9
3. Specific Requirements.....	10
3.1 External Interface Requirements.....	10
3.1.1 User Interfaces.....	10
3.1.2 Hardware Interfaces	10
3.1.3 Software Interfaces.....	10
3.1.4 Communications Interfaces.....	10
3.2 System Features.....	10
3.2.1 Player Identification.....	10
3.2.1.1 Description and Priority.....	10
3.2.1.2 Pre-condition	11
3.2.1.3 Post-condition	11
3.2.1.4 Stimulus/Response Sequence	11
3.2.1.5 Functional Requirements	11
3.2.2 Matchmaking Queue	11
3.2.2.1 Description and Priority.....	11
3.2.2.2 Pre-condition	11
3.2.2.3 Post-condition	11
3.2.2.4 Stimulus/Response Sequence	11
3.2.2.5 Functional Requirements	12
3.2.3 Champion Selection.....	12
3.2.3.1 Description and Priority.....	12
3.2.3.2 Pre-condition	12
3.2.3.3 Post-condition	12
3.2.3.4 Stimulus/Response Sequence	12
3.2.3.5 Functional Requirements	12
3.2.4 Real-Time Combat	13
3.2.4.1 Description and Priority.....	13
3.2.4.2 Pre-condition	13
3.2.4.3 Post-condition	13
3.2.4.4 Stimulus/Response Sequence	13

3.2.4.5 Functional Requirements	13
3.2.5 Match Win Conditions	15
3.2.5.1 Description and Priority	15
3.2.5.2 Pre-condition	15
3.2.5.3 Post-condition	15
3.2.5.4 Stimulus/Response Sequence	15
3.2.5.5 Functional Requirements	15
3.2.6 Disconnect and Reconnect Handling	15
3.2.6.1 Description and Priority	15
3.2.6.2 Pre-condition	15
3.2.6.3 Post-condition	16
3.2.6.4 Stimulus/Response Sequence	16
3.2.6.5 Functional Requirements	16
3.2.7 Match History	16
3.2.7.1 Description and Priority	16
3.2.7.2 Pre-condition	16
3.2.7.3 Post-condition	16
3.2.7.4 Stimulus/Response Sequence	16
3.2.7.5 Functional Requirements	17
3.2.8 Leaderboard	17
3.2.8.1 Description and Priority	17
3.2.8.2 Pre-condition	17
3.2.8.3 Post-condition	17
3.2.8.4 Stimulus/Response Sequence	17
3.2.8.5 Functional Requirements	17
3.3 Performance Requirements	17
3.4 Logical Database Requirements	18
3.5 Design Constraints	18
3.6 Software System Attributes	18
3.6.1 Reliability	18
3.6.2 Robustness	18
3.6.3 Availability	18
3.6.4 Maintainability	19
3.6.5 Usability	19
3.6.6 Security	19
3.6.7 Portability	19
4. References and Sources of Information	20
Appendix A — WebSocket Contract Summary	20
Client → Server events	20
Server → Client events	20
Appendix B — Champion Roster Summary	21
Appendix C — AI-Use Plan	21
Index	23

Authorship — Primary Responsibility by Section

Per course instructions, this table annotates the Table of Contents above to indicate which team member had primary responsibility for each section's content, consistent with the division of labor in the team's Project Plan (Marshall: server/real-time/testing/deployment; Raj: client application; En: game design/data layer/persistence/documentation). Sections whose content is genuinely joint list more than one name.

Section	Primary Responsibility
1. Introduction	En
2. Overall Description	En
3.1 External Interface Requirements	Raj
3.2.1 Player Identification	En
3.2.2 Matchmaking Queue	Marshall
3.2.3 Champion Selection	Marshall (server logic); En (champion data); Raj (UI)
3.2.4 Real-Time Combat	Marshall (server logic); Raj (client rendering)
3.2.5 Match Win Conditions	Marshall
3.2.6 Disconnect and Reconnect Handling	Marshall
3.2.7 Match History	En
3.2.8 Leaderboard	En
3.3 Performance Requirements	Marshall
3.4 Logical Database Requirements	En
3.5 Design Constraints	Marshall
3.6 Software System Attributes	Marshall (3.6.1–3.6.4, 3.6.6–3.6.7); Raj (3.6.5 Usability)
4. References	En
Appendix A — WebSocket Contract Summary	Marshall
Appendix B — Champion Roster Summary	En
Appendix C — AI-Use Plan	En
Index	En

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for Arena, a browser-based real-time multiplayer combat game developed as a term project for this course. This document is intended to serve as the agreed-upon description, between the course instructor (acting as customer) and the development team, of what the Arena system will do and the quality attributes it must exhibit. Every requirement in Section 3 is written to be verifiable: each one is written so that it can be verified by an automated test, a manual demonstration, or a direct code inspection once implemented.

The intended audience for this document is the course instructor, the three members of the development team, and any future maintainer of the Arena codebase.

1.2 Problem Statement

Real-time multiplayer games are technically demanding to build correctly: they require an authoritative server that can resolve simultaneous player actions consistently, synchronize fast-changing state to multiple clients with minimal latency, and remain correct under network conditions a single-player or request/response web application never has to handle — dropped connections, out-of-order messages, and two players attempting conflicting actions in the same instant. Most existing approaches to this problem are disproportionate to a small class project: standing up dedicated game-server infrastructure, a matchmaking backend, and persistent player accounts is normally the work of a much larger team over a much longer timeline.

At the same time, from a player's perspective, the simplest version of this problem — two people who want to play a quick real-time match against each other — is often made harder than it needs to be. LAN-only solutions require both players to be on the same network; many hobby or indie multiplayer projects require a manual, technical setup step, such as port forwarding or exchanging a shared IP address, before two people on different networks can connect at all.

Arena is scoped to address both problems together: it demonstrates, within a three-week team project, the core engineering challenges of a real-time authoritative multiplayer system — server-authoritative combat resolution, live state synchronization, matchmaking, and disconnect handling — while remaining simple enough that two players, on two different computers and two different internet connections, can reach a live match through nothing more than a web browser and a shared link.

1.3 Scope

The software product to be produced is named Arena. Arena allows two players to compete against each other in real time, each controlling a champion selected from a small, fixed roster, inside a single arena instance. Arena is composed of three subsystems:

- A browser-based client application (React/TypeScript) that a player uses to identify themselves, queue for a match, select a champion, view live match state, and view match results, personal match history, and a leaderboard.

- An authoritative real-time game server (Node.js/TypeScript, WebSocket via Socket.IO) that owns every match-critical decision: matchmaking, movement, ability resolution, damage, cooldowns, and win conditions.
- A REST API and PostgreSQL persistence layer that stores player profiles, records completed match results, and serves match history and leaderboard data to the client.

Arena, in this version, will not include: skill-based matchmaking (a simple first-in-first-out queue is used instead); more than two players per match (team-based or larger matches are out of scope); authentication beyond a client-supplied username (no passwords, OAuth, or other credentialed login); monetization or payment features; anti-cheat or fraud detection beyond server-side validation of every player action; or voice/text chat between players.

The goals of the Arena project are to demonstrate (a) a functionally correct, reliable, real-time multiplayer system in which the server is the sole source of truth for game outcomes; (b) an automated, repeatable testing and deployment pipeline; and (c) a maintainable, modular architecture suitable for a small team where each member owns a well-defined subsystem.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
Arena	The software system described by this document; also the name of the single combat area a match takes place in.
Champion	A selectable in-game character with a fixed set of stats and abilities.
Match	A single instance of gameplay between two players, from champion selection through a win condition being met.
Tick	One discrete simulation step of the server's authoritative game loop, occurring 20 times per second.
Ability	A champion-specific action with a cooldown, a resource cost, and one effect type: damage, heal, crowd control, or positioning.
Cooldown	The minimum time that must elapse between two uses of the same ability by the same player.
Crowd Control (CC)	An ability effect that temporarily prevents the target from moving or using abilities.
Matchmaking Queue	The first-in-first-out list of identified players waiting to be paired into a match.
Disconnect Grace Period	The interval during which a disconnected player's champion is held in place, pending reconnection, before the match resolves as a loss for that player's team.
Player	A person using the Arena client, identified by a persistent player identifier and a chosen username.
Client	The React/TypeScript browser application used by a player.
Server / Game Server	The authoritative Node.js/TypeScript process running match simulation, communicating with clients over WebSocket.
API / REST API	The Node.js/Express service exposing player profile, match history, and leaderboard data over HTTP.
WebSocket Contract	The shared, versioned set of TypeScript interfaces defining every message exchanged between client and server.
Win Condition	One of: elimination, time limit, or disconnect forfeit — see Section 3.2.5.

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — the project's automated test-and-deploy pipeline.

1.5 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, approved 25 June 1998.

[2] Wiegers, Karl S. "Writing Quality Requirements." Course-supplied reading.

[3] Humphrey, Watts S. "The Personal Software Process: Overview, Practice and Results" and "The Personal Software Process." Course-supplied readings.

[4] Arena project one-pager and Team Project Plan, provided by the course instructor / prepared by the team.

[5] Arena source code repository: <https://github.com/Blotjarl/arena>

[6] Socket.IO documentation: <https://socket.io/docs/>

[7] PostgreSQL documentation: <https://www.postgresql.org/docs/>

1.6 Overview

The remainder of this document is organized as follows. Section 2 (Overall Description) gives background on Arena's product perspective, its major functions, its intended users, and the constraints and assumptions under which it is being built, without stating specific requirements. Section 3 (Specific Requirements) is the core of the document: it states verifiable functional requirements organized by system feature — each with a description and priority, pre/post-conditions, a stimulus/response sequence, and numbered “shall” requirements — followed by external interface requirements, performance requirements, design constraints, and non-functional (quality) requirements. Section 4 lists references. The appendices provide supplementary material, including a summary of known gaps for the final revision of this document.

2. Overall Description

2.1 Product Perspective

Arena is a new, self-contained software system; it does not replace, and is not a component of, any pre-existing system. It is composed of three independently deployable subsystems communicating over well-defined interfaces: a browser client, a real-time game server, and a REST API backed by a PostgreSQL database. At a high level, the client communicates with the game server over a persistent WebSocket connection for everything related to live gameplay, and with the REST API over ordinary HTTP requests for match history and leaderboard data. The game server, in turn, notifies the persistence layer when a match starts and ends, so match results can be recorded without the game server itself performing direct database writes on the hot path of match simulation.

The single most important architectural property of Arena is that the game server is the sole source of truth for every match-critical decision. The client is treated only as a display and input device: it renders whatever state the server sends and forwards player input as requests, but it never asserts an outcome — such as damage dealt, a cooldown having elapsed, or a match having been won — that the server has not itself computed and broadcast.

2.2 Product Functions

The following is a summary of Arena's major functions; each is stated as a verifiable requirement in Section 3.

- Player identification by username, with no password.
- A matchmaking queue: join, cancel, first-in-first-out pairing, capacity-limited concurrent matches.
- Champion selection from a fixed roster of three champions.
- Real-time combat: movement, ability use, damage/heal/crowd-control/positioning effects, cooldowns, and resource management.
- Match win-condition resolution: elimination, time limit, and disconnect forfeit.
- Disconnect and reconnect handling with a grace period.
- Retrieval of a player's match history.
- A leaderboard ranking players by win rate, and a summary of win rates per champion.

2.3 User Characteristics

The intended end users of Arena are casual players with basic familiarity operating a mouse, a keyboard, and a modern web browser. No specialized gaming knowledge, account creation beyond choosing a username, or software installation is required. A secondary reader of parts of this system (though not a user of the software itself) is the course instructor, who evaluates the system against this SRS and against the course's expectations of software engineering practice.

2.4 Operating Environment

Client: any evergreen desktop web browser (e.g., Chrome, Firefox, Edge, Safari) supporting WebSocket and modern JavaScript (ECMAScript 2022); no browser plugin or native application install is required.

Server and API: a Node.js 20+ runtime, to be packaged as Docker container images. The production deployment target is Railway (railway.app), a platform-as-a-service that builds and runs the project's existing Docker images directly and provisions a managed PostgreSQL instance; each deployed service receives a public HTTPS URL reachable from any network, which is what allows two players on different computers and different internet connections to play together without local network configuration. The containerized design does not depend on any Railway-specific runtime feature — see 3.6.7, Portability.

Persistence: PostgreSQL 16.

2.5 Constraints

- A fixed, three-week development timeline set by the course.
- A team of three students, each individually responsible for one architectural layer (server/real-time/testing/deployment; client; game design/data/persistence).
- No hosting is provided by the department; any live deployment cost is borne by the team, which is why a low-cost, usage-billed platform-as-a-service (Railway) was chosen over a full custom cloud infrastructure buildout.
- AI-assisted development (Claude Code) is planned per the course's disclosed tooling policy; an AI-use plan is provided (see Appendix C).
- The system must be demonstrable running entirely locally via Docker, without requiring graders to have cloud infrastructure access.

2.6 Assumptions and Dependencies

- Players have a reasonably stable internet connection; the disconnect-handling requirements in 3.2.6 assume transient, not permanent, network loss.
- This version of Arena is scoped to exactly two players per match (1v1); team-based or larger matches are not required.
- Third-party runtime dependencies (Node.js, Socket.IO, React, PostgreSQL, and the npm packages built on them) remain available and compatible with the versions pinned in the project's package manifests for the duration of the project.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The client shall present the following screens:

- A Lobby screen with a username entry field and a “Find Match” control, and a queue status/cancel control once queued.
- A Champion Select screen showing both competing players, a selection countdown timer, and the available champion roster with each champion's stats and abilities.
- An in-Match HUD showing both players' health and resource bars, ability cooldown indicators, and a rendering of the arena with both champions' live positions.
- A Results screen showing the match outcome (win/loss/draw), the reason the match ended, its duration, and a control to return to the queue.

3.1.2 Hardware Interfaces

None beyond a standard keyboard (arrow keys or WASD for movement input) and mouse (menu interaction) attached to the client device. No specialized hardware is required.

3.1.3 Software Interfaces

- Socket.IO (client and server libraries) for the real-time WebSocket transport.
- PostgreSQL, via the pg Node.js driver, for persistent storage, accessed only by the game server and REST API processes.
- Express.js for the REST API's HTTP routing.

3.1.4 Communications Interfaces

All real-time client-server communication uses the WebSocket protocol via Socket.IO, per the interfaces defined in the project's shared WebSocket Contract. All client-API communication uses HTTP/REST with JSON request and response bodies.

3.2 System Features

This section organizes Arena's functional requirements by feature — the tasks a user can accomplish with the system — per IEEE Std 830-1998, 5.3.7.4. Each feature is stated as a description and priority, applicable pre/post-conditions, a representative stimulus/response sequence, and a set of numbered, verifiable “shall” requirements.

3.2.1 Player Identification

3.2.1.1 Description and Priority

The system shall allow a person to identify themselves to Arena by supplying a username, without a password.

Priority: Essential

3.2.1.2 Pre-condition

The user has loaded the Arena client in a browser and has not yet entered a username.

3.2.1.3 Post-condition

The user has a stable player identifier associated with their chosen username for the remainder of the session.

3.2.1.4 Stimulus/Response Sequence

1. The user enters a username into the Lobby screen's username field.
2. The user submits the form (e.g., by clicking “Find Match”).
3. The client generates, or reuses if already present for this browser session, a persistent player identifier, and sends an identify message to the server containing that identifier and the username.
4. The system proceeds to the Matchmaking Queue feature (3.2.2).

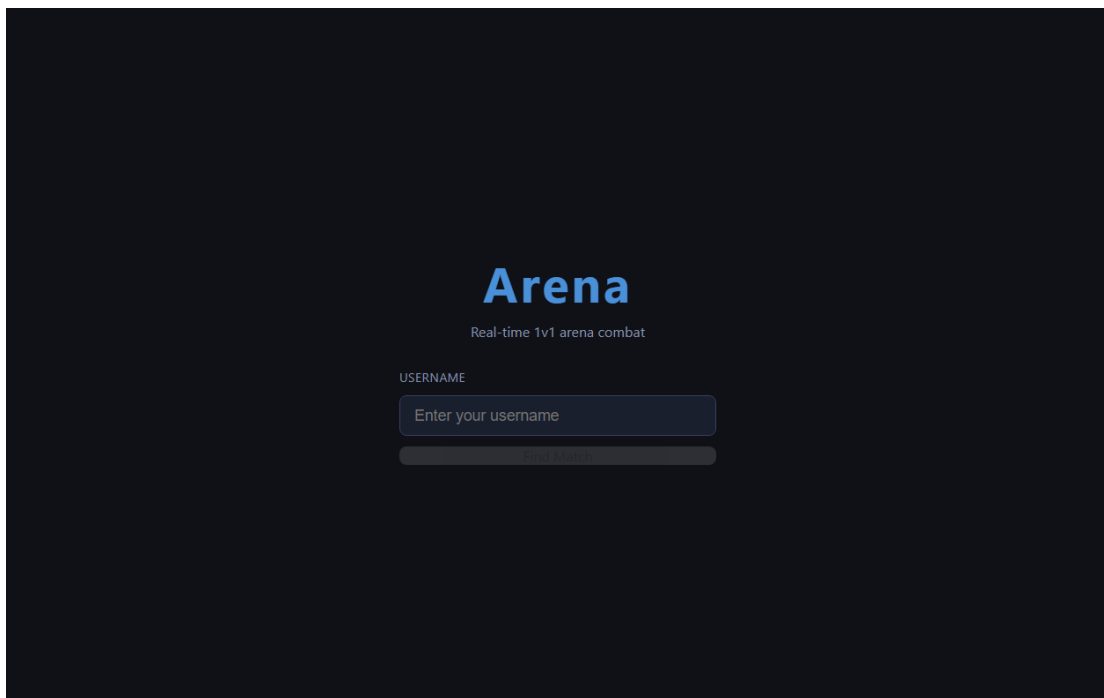
3.2.1.5 Functional Requirements

R1.1 The system shall require a non-empty username of no more than 24 characters before allowing a user to queue for a match.

R1.2 The system shall generate a player identifier that persists for the duration of the browser session, so a user who reloads the page retains the same identifier.

R1.3 The system shall not require a password or any other credential to identify a player.

R1.4 The system shall reject, with an error indicating the reason, any WebSocket message from a client that has not first sent a valid identify message.



3.2.2 Matchmaking Queue

3.2.2.1 Description and Priority

The system shall allow an identified player to join a first-in-first-out queue to be matched against another player, and to cancel a pending queue request.

Priority: Essential

3.2.2.2 Pre-condition

The player has completed Player Identification (3.2.1).

3.2.2.3 Post-condition

The player is either matched with an opponent (3.2.3) or has returned to an unqueued state.

3.2.2.4 Stimulus/Response Sequence

5. The player requests to join the queue.
6. The system acknowledges the request and reports the player's queue position.
7. The system pairs the player with the next available opponent once both a match slot and an opponent are available, and notifies both paired players; or the player requests to cancel, and the system removes them from the queue and confirms cancellation.

3.2.2.5 Functional Requirements

R2.1 The system shall add a requesting, identified player to a first-in-first-out matchmaking queue and report their 1-based position in that queue.

R2.2 The system shall reject a queue-join request from a player who is already queued or already in an active match.

R2.3 The system shall allow a queued player to cancel their queue request at any time before being matched.

R2.4 The system shall pair the two longest-waiting queued players into a new match whenever a match slot is available.

R2.5 The system shall enforce a configurable upper bound on the number of concurrent matches; requests beyond that bound shall remain queued, in order, until a match slot is freed by a concluded match.

R2.6 The system shall notify both paired players, within one second of pairing, that a match has been found, including each player's assigned team and their opponent's identity.

3.2.3 Champion Selection

3.2.3.1 Description and Priority

The system shall allow each of the two matched players to select one champion from the available roster before the match begins.

Priority: Essential

3.2.3.2 Pre-condition

The player has been matched (3.2.2) and has not yet selected a champion for this match, and the match's champion-selection window has not yet expired.

3.2.3.3 Post-condition

Both players have selected champions and the match has begun (3.2.4), or the selection window has expired and the match has ended without beginning.

3.2.3.4 Stimulus/Response Sequence

8. The system presents both players with the available champion roster, each entry showing the champion's stats and abilities.
9. Each player selects a champion.
10. The system broadcasts each selection to both players as it occurs, along with whether all players have now selected.
11. Once both players have selected, the system begins the match and sends both players the initial match state.

3.2.3.5 Functional Requirements

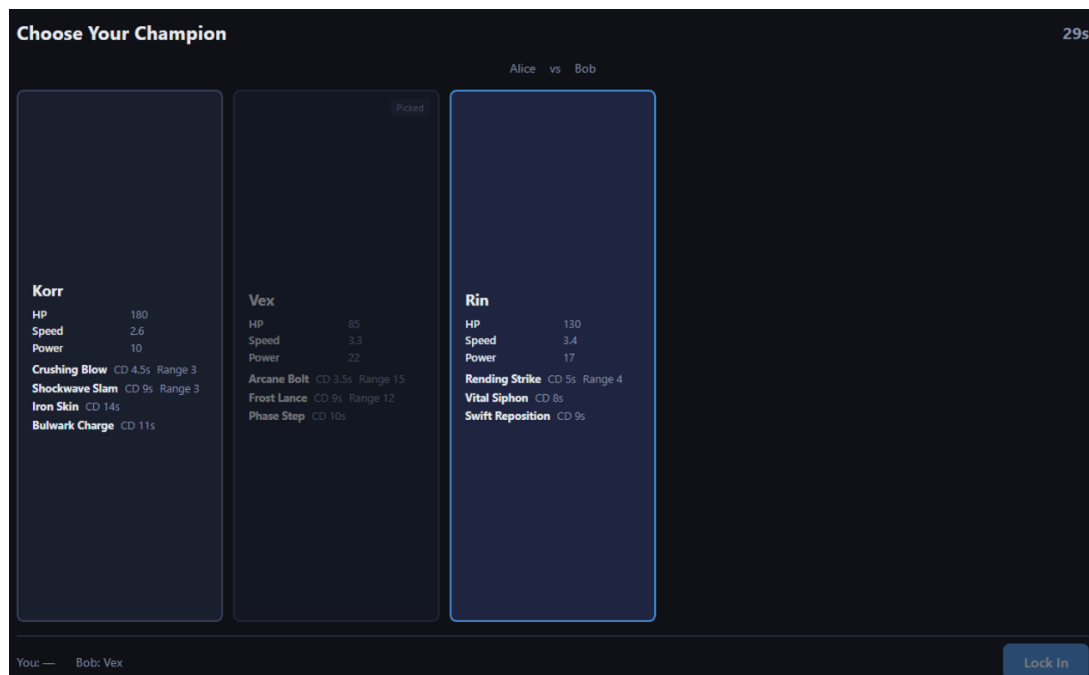
R3.1 The system shall present the same, fixed roster of champions to both matched players.

R3.2 The system shall reject a champion-selection request naming a champion not present in the roster.

R3.3 The system shall broadcast a player's champion selection to their opponent as it occurs.

R3.4 The system shall enforce a 30-second champion-selection time limit; if either player has not selected a champion when the limit is reached, the system shall end the match without it beginning and record the reason as a selection timeout.

R3.5 The system shall begin the match, and transition both players to the in-Match HUD, immediately once both players have selected a champion.



3.2.4 Real-Time Combat

3.2.4.1 Description and Priority

The system shall allow each player, once a match has begun, to move their champion and use that champion's abilities, with all outcomes computed authoritatively by the server.

Priority: Essential

3.2.4.2 Pre-condition

Both players have completed Champion Selection (3.2.3) and the match is active.

3.2.4.3 Post-condition

The match's state (positions, health, resource, cooldowns) reflects all validly submitted player actions since the match began.

3.2.4.4 Stimulus/Response Sequence

12. A player submits a movement direction, or an ability-use request naming the ability and, if applicable, a target player or position.
13. The system validates the request against the champion's current state: cooldown remaining, resource available, range to target, whether the caster is alive and not under crowd control.
14. If valid, the system applies the action's effect — movement integration, damage, healing, crowd control, or repositioning — to its authoritative match state.
15. If invalid, the system silently ignores the request without applying any effect.
16. At each simulation tick (20 times per second), the system broadcasts the current match state to both players.

3.2.4.5 Functional Requirements

R4.1 The system shall move a champion only in response to a movement request from that champion's own player, at a speed determined by the champion's stats, never trusting a client-reported position.

R4.2 The system shall apply an ability's damage, healing, crowd-control, or positioning effect only after validating that the ability is off cooldown, the caster has sufficient resource, the caster is alive and not under crowd control, and, for targeted abilities, the target is within the ability's range.

R4.3 The system shall set an ability's cooldown, and deduct its resource cost, at the moment the ability is validly used.

R4.4 The system shall regenerate each champion's resource continuously over time at that champion's stat-defined rate, up to that champion's maximum.

R4.5 The system shall broadcast the authoritative match state — all players' positions, health, resource, and remaining cooldowns — to both players at the server's fixed simulation rate.

R4.6 The system shall resolve two players' actions submitted for the same simulation tick in the order they were received by the server, such that an earlier crowd-control effect can prevent a later action by the affected player within that same tick.

R4.7 The client shall render champion movement smoothly between authoritative server updates using client-side interpolation, without altering or overriding any authoritative value (health, resource, cooldowns) received from the server.

3.2.5 Match Win Conditions

3.2.5.1 *Description and Priority*

The system shall end an active match, and record its outcome, once one of the defined win conditions is met.

Priority: Essential

3.2.5.2 *Pre-condition*

A match is active (3.2.4).

3.2.5.3 *Post-condition*

The match is marked ended, both players have been notified of the outcome, and, if the match had begun, the result is queued for persistence.

3.2.5.4 *Stimulus/Response Sequence*

17. At each simulation tick, the system checks whether a win condition has been met.
18. If a player's champion's health has reached zero, the system ends the match, crediting the win to the opposing team.
19. If the match's time limit has been reached without elimination, the system ends the match, crediting the win to whichever player has more remaining health, or declaring a draw if equal.
20. The system notifies both players of the match's outcome, reason, and duration, then returns both players to the lobby.

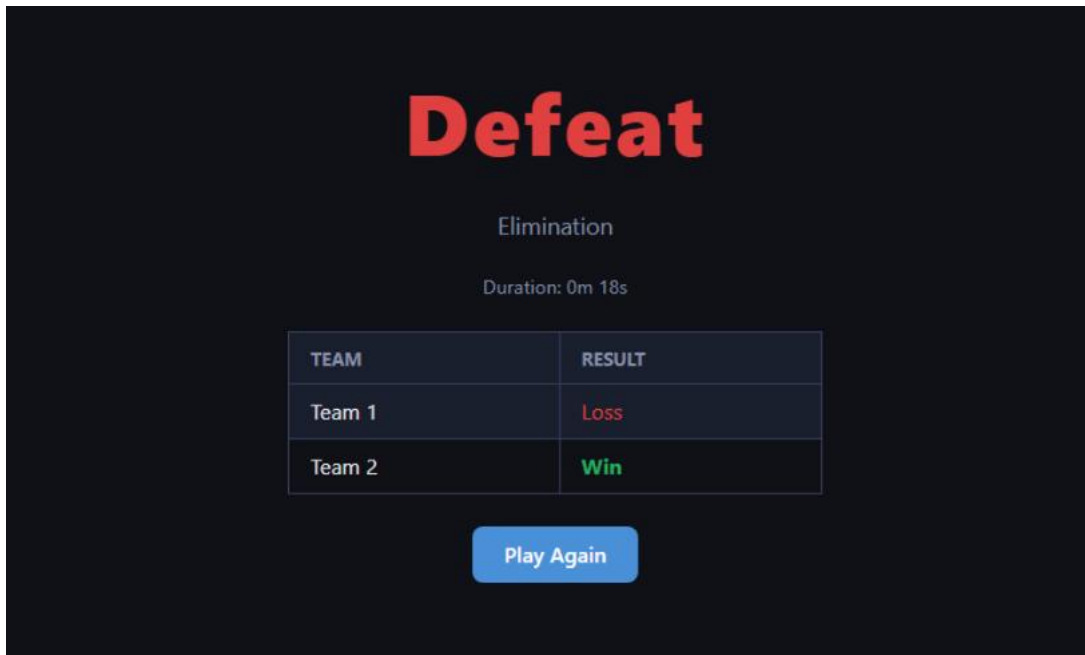
3.2.5.5 *Functional Requirements*

R5.1 The system shall end a match by elimination immediately upon a champion's health reaching zero, crediting the win to the opposing player.

R5.2 The system shall end a match by time limit if neither champion has been eliminated within five minutes of the match beginning, crediting the win to the player with the higher remaining health, or declaring a draw if health is equal.

R5.3 The system shall report, for every ended match, the reason it ended (elimination, time limit, disconnect forfeit, or selection timeout), its duration, and each player's result.

R5.4 The system shall not corrupt or otherwise affect the state of any other in-progress match when one match ends.



3.2.6 Disconnect and Reconnect Handling

3.2.6.1 Description and Priority

The system shall tolerate a player's transient network disconnection during an active match without immediately forfeiting the match, while still resolving the match if the disconnection is not resolved in time.

Priority: Essential

3.2.6.2 Pre-condition

A match is active and one player's connection to the server is lost.

3.2.6.3 Post-condition

Either the player has reconnected and play continues, or the disconnect grace period has elapsed and the match has ended as a forfeit for the disconnected player's team.

3.2.6.4 Stimulus/Response Sequence

21. The server detects that a player's connection has been lost.
22. The system holds the disconnected player's champion in place and notifies the remaining player of the disconnection and the remaining grace period.
23. If the disconnected player reconnects and re-identifies within the grace period (30 seconds), the system restores their connected status, notifies both players, and play continues with no other state affected.
24. If the grace period elapses without reconnection, the system ends the match as a forfeit, crediting the win to the opposing team.

3.2.6.5 Functional Requirements

R6.1 The system shall hold a disconnected player's champion stationary — no further movement or ability use on their behalf — for the duration of the disconnect grace period.

R6.2 The system shall notify the remaining player of a disconnection and the remaining grace period at the moment the disconnection is detected.

R6.3 The system shall restore a reconnecting player's full participation in the match, with match state unaffected for the remaining player, provided reconnection occurs within the grace period.

R6.4 The system shall resolve the match as a disconnect forfeit, crediting the opposing team, if the disconnect grace period (30 seconds) elapses without reconnection.

3.2.7 Match History

3.2.7.1 Description and Priority

The system shall record every completed match that began (i.e., reached completion of Champion Selection) and make a player's match history available on request.

Priority: Essential (recording); Desired (client-side history view)

3.2.7.2 Pre-condition

A match has ended after having begun (3.2.5); a match that ended before both players selected a champion is not recorded.

3.2.7.3 Post-condition

The match's result, participants, champions played, reason, and duration are retrievable via the REST API.

3.2.7.4 Stimulus/Response Sequence

25. The game server notifies the persistence layer that a match has begun, capturing each player's chosen champion, and, separately, that it has ended, capturing the outcome.
26. The persistence layer correlates the two notifications and writes one match record and one participant record per player to the database.
27. A client requests a specific player's match history via the REST API.
28. The system returns that player's completed matches, most recent first, including each match's outcome, champion played, opponent, and duration.

3.2.7.5 Functional Requirements

R7.1 The system shall persist, for every match that began, each participating player, their chosen champion, their result (win/loss/draw), the match's end reason, and its duration.

R7.2 The system shall not persist a match that ended before both players selected a champion (a selection-timeout match).

R7.3 The system shall provide a REST endpoint returning a specified player's match history, ordered from most to least recent, supporting pagination.

R7.4 A failure to persist a completed match shall be logged and shall not interrupt or crash the live game server process, nor affect any in-progress match.

3.2.8 Leaderboard

3.2.8.1 Description and Priority

The system shall make available a ranking of players by win/loss record, and a summary of win rates per champion.

Priority: Desired

3.2.8.2 Pre-condition

At least one match has been persisted (3.2.7).

3.2.8.3 Post-condition

An up-to-date leaderboard, reflecting all persisted matches, is retrievable via the REST API.

3.2.8.4 Stimulus/Response Sequence

29. A client requests the leaderboard via the REST API.
30. The system computes, from persisted match history, each qualifying player's wins, losses, draws, games played, and win rate.
31. The system returns players ranked by win rate, with ties broken by total wins.

3.2.8.5 Functional Requirements

- R8.1** The system shall compute each player's win rate as wins divided by games played, derived from persisted match history rather than a separately maintained running total.
- R8.2** The system shall exclude, from the leaderboard, players with fewer than a configurable minimum number of games played (default: 1).
- R8.3** The system shall provide a REST endpoint returning champion win rates aggregated across all players and matches.

3.3 Performance Requirements

- R-P1** The game server shall advance its authoritative simulation at a fixed rate of 20 ticks per second.
- R-P2** The system shall broadcast a match-state update to both players within one simulation tick (50 ms) of the state changing.
- R-P3** The system shall enforce a configurable upper bound on concurrently active matches (default: 50) to bound per-instance server load.
- R-P4** The client shall render champion movement at a smooth, consistent frame rate on a modern desktop browser, using interpolation to compensate for the gap between discrete server updates.
- R-P5** The REST API shall respond to a match-history or leaderboard request in under one second under normal load.

3.4 Logical Database Requirements

This subsection specifies the logical structure of the data persisted by the system (3.2.7, 3.2.8), independent of its physical storage. The persistence layer maintains three entities:

- **Player** — one record per unique username, holding a stable player identifier, the username, and the timestamp the profile was first created. A player record is created automatically the first time that username is seen by the system (3.2.1); no separate registration step exists.
- **Match** — one record per match that reached Champion Selection completion (3.2.7.2), holding a stable match identifier, the reason the match ended, the winning team (nullable, for a draw), the match's duration, and the timestamp it ended.
- **Match Participant** — one record per player per match, linking a Player to a Match, holding that player's team, the champion they played, and their result (win, loss, or draw). This is the entity that makes match history (3.2.7) and the leaderboard (3.2.8) queryable per player and per champion.

A Match has exactly two associated Match Participant records (one per team, per the 1v1 scope in 2.6); a Match Participant belongs to exactly one Match and references exactly one Player.

R-DB1 The system shall persist each unique player exactly once, keyed by a stable player identifier, regardless of how many matches that player completes.

R-DB2 The system shall persist each match that reached Champion Selection completion exactly once, together with exactly one match-participant record per player in that match.

R-DB3 The system shall enforce, at the data layer, that a match's recorded reason is one of the four values defined in 3.2.5 (elimination, time limit, disconnect forfeit, selection timeout), and that a match-participant's recorded result is one of win, loss, or draw.

R-DB4 The system shall enforce referential integrity between Match Participant records and their associated Match and Player records, such that a match-participant record cannot exist without both.

R-DB5 The system shall support efficient retrieval of all match-participant records for a given player (for match history, 3.2.7) and all match-participant records for a given champion (for champion win rates, 3.2.8), without scanning the full match history table.

R-DB6 The system shall retain match and player history indefinitely; no automatic deletion or expiration of match records is required in this version.

3.5 Design Constraints

R-D1 The system shall be implemented in TypeScript across all subsystems (client, server, API).

R-D2 The system shall use Socket.IO over WebSocket for all real-time client-server communication, per the versioned, shared WebSocket Contract.

R-D3 The system shall use PostgreSQL as its persistent data store.

R-D4 The system shall be containerized with Docker such that each subsystem (game server, REST API) can be built and run as an independent container image.

R-D5 The system's automated test suite (Jest for unit tests, Playwright for end-to-end tests) shall run in a continuous integration pipeline on every change proposed to the shared code repository.

R-D6 This document shall follow the outline and content guidance of IEEE Std 830-1998.

R-D7 The system shall be reachable over the public internet via a stable URL per deployed service, such that two players on different computers connected to different internet service providers can

join the same match without a shared local network or manual network configuration (e.g., port forwarding).

3.6 Software System Attributes

3.6.1 Reliability

The game server shall be the sole authority for all match-critical outcomes — damage, cooldowns, and win conditions; the system shall not accept a client-asserted value for any of these. A failure in the persistence layer (3.2.7, 3.2.8) shall not cause the loss of an in-progress match's live state, nor interrupt gameplay for either participant.

3.6.2 Robustness

The system shall validate every player-submitted action against the current authoritative match state before applying any effect, and shall reject, without effect, any action that fails validation (e.g., insufficient resource, ability on cooldown, target out of range, acting player not part of the named match). The system shall continue normal operation for all other active matches if any single match encounters an unexpected internal error.

3.6.3 Availability

The system shall support redeploying the game server, for example to release a new version, without corrupting the state of matches active at the moment of redeployment beyond what is already specified for disconnect handling in 3.2.6.

3.6.4 Maintainability

The system's server-side game logic (match state, tick loop, ability resolution, matchmaking) shall be implemented independently of the network/WebSocket transport layer, such that it can be exercised by automated tests without a live network connection. The system shall maintain automated unit test coverage for match state transitions, ability resolution, matchmaking, and disconnect/reconnect handling, and at least one automated end-to-end test covering a complete match from connection through match end.

3.6.5 Usability

The client shall present clear, human-readable feedback for every rejected action or error condition (for example, an invalid champion selection), rather than failing silently from the user's perspective. A first-time user shall be able to reach an active match — identify, queue, select a champion — without requiring instructions beyond the on-screen prompts.

3.6.6 Security

The system's identity model is limited by design to a client-supplied username with no password (see 1.2, Scope); the system shall not claim or imply a stronger identity guarantee than this. The system shall not store or transmit any credential-equivalent secret for players. Database credentials and other deployment secrets shall not be committed to the source code repository.

3.6.7 Portability

The system's containerized subsystems (game server, REST API) shall be deployable on any container orchestration platform supporting standard Docker images, and shall not depend on a proprietary, non-portable platform-specific runtime feature; the deployment's current use of a managed platform-as-a-service (Railway) for compute and PostgreSQL hosting is a deployment choice, not an architectural requirement of the software itself, and the system could be redeployed to another container host (for example Render, Fly.io, or a self-managed server) without changes to the application code.

4. References and Sources of Information

The following sources were used in the preparation of this document and the Arena system:

- [1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, approved 25 June 1998.
- [2] Wiegers, Karl S. "Writing Quality Requirements." Course-supplied reading.
- [3] Humphrey, Watts S. "The Personal Software Process: Overview, Practice and Results" (1996) and "The Personal Software Process" (2000). Course-supplied readings, referenced for engineering-notebook and verifiability practices.
- [4] Humphrey, Watts S. Introduction to the Team Software Process. Addison-Wesley, 2000 (sample SRS content outline, p. 113, provided by the course instructor).
- [5] Arena Game Project Description (one-pager) and Arena Team Project Plan — internal project documents prepared by the team and course instructor.
- [6] Arena source code repository: <https://github.com/Blotjarl/arena>
- [7] Socket.IO documentation: <https://socket.io/docs/>
- [8] PostgreSQL documentation: <https://www.postgresql.org/docs/>
- [9] React documentation: <https://react.dev/>
- [10] Four example course SRS documents reviewed for format and content guidance: "Brewhouse Fermentation System SRS" (CS 4398 Group 3), "Android Weather Application SRS" (CS 3398 Group 1), "Classified Ads Website SRS" (CS 5394), and "Slither: A Game of Strategy SRS."

Appendix A — WebSocket Contract Summary

The full, authoritative contract will be defined as TypeScript source in the project's shared package (packages/shared/src/events.ts); this appendix summarizes the planned contract for reference. Every event below will be exchanged over a single persistent WebSocket connection per client.

Client → Server events

Event	Purpose
identify	Associate this connection with a player identifier and username (3.2.1).
queue:join	Join the matchmaking queue (3.2.2).
queue:cancel	Leave the matchmaking queue (3.2.2).
champion:select	Select a champion for the current match (3.2.3).
match:action	Submit a movement or ability-use request during an active match (3.2.4).
match:reconnect	Re-associate a new connection with an in-progress match after a disconnect (3.2.6).

Server → Client events

Event	Purpose
queue:joined	Acknowledges a queue-join request with the player's position.

Event	Purpose
queue:cancelled	Acknowledges a successful queue cancellation.
match:found	Notifies both players that they have been paired, with team assignment and the champion roster.
champion:selected	Broadcasts a champion selection and whether both players have now selected.
match:start	Sends the initial authoritative match state once both players have selected.
match:state	Broadcasts the current authoritative match state, sent every simulation tick.
match:end	Reports the match outcome, reason, duration, and each player's result.
match:player_disconnected	Notifies the remaining player of an opponent's disconnection and grace period.
match:player_reconnected	Notifies both players that a disconnected player has reconnected.
error	Reports a rejected request and a machine-readable reason code.

Appendix B — Champion Roster Summary

The planned roster will consist of three champions, each filling a distinct role:

Champion	Role	Max Health	Notable Abilities
Korr	Bruiser / Control	180	Crushing Blow (melee damage), Shockwave Slam (stun), Iron Skin (self-heal), Bulwark Charge (gap-closer)
Vex	Ranged Burst Mage	85	Arcane Bolt (long-range damage), Frost Lance (ranged root), Phase Step (blink)
Rin	Sustain Duelist	130	Rending Strike (melee damage), Vital Siphon (self-heal), Swift Reposition (gap-closer/disengage)

Each champion will have at least one damage-dealing ability, since there is no auto-attack system; a kit without a damage ability could never win by elimination. Full stats and ability parameters (cooldowns, resource costs, ranges, magnitudes) will be maintained in a shared champion-kit source file, which will serve as the authoritative reference.

Appendix C — AI-Use Plan

Per the course's tooling policy, the team plans to use Claude Code (an AI coding agent) throughout development. The following describes how AI assistance is planned to be used, and independently verified, for each subsystem; a fuller log with per-commit detail will be maintained in the project repository as development proceeds.

Area	Planned AI Involvement
Shared WebSocket contract (packages/shared)	To be AI-generated from a written specification, then reviewed and iterated by Marshall.
Game server (match logic, matchmaking, tick loop)	To be AI-generated, then reviewed, tested, and debugged by Marshall.

Area	Planned AI Involvement
Client application (packages/client)	To be AI-generated by Raj's agent, then reviewed and integration-tested by Marshall before merging.
Champion roster and persistence layer (packages/api)	To be AI-generated by En's agent, then reviewed and integration-tested by Marshall before merging.
Test suites (Jest, Playwright)	To be AI-generated alongside each corresponding piece of functionality; to be verified by actually running the suites, not merely inspecting the code.
Docker, Terraform, CI/CD configuration	To be AI-generated, then verified by performing real local Docker builds and, where applicable, real deployments.
This SRS document	AI-generated preliminary draft, produced from the course's SRS instructions, IEEE Std 830-1998, and four example SRS documents. To be reviewed and finalized by the team.

Index

This index references section numbers rather than page numbers; page numbers will be added once the document's final layout is set.

Term	Section(s)
Ability	1.4; 3.2.4
AI-Use Plan	Appendix C
Authorship Table	p. iii
Champion	1.4; 3.2.3; Appendix B
Champion Selection	3.2.3
Client	1.4; 2.4; 3.1.1
Constraints	2.5; 3.5
Cooldown	1.4; 3.2.4
Crowd Control (CC)	1.4; 3.2.4
Design Constraints	3.5
Disconnect Grace Period	1.4; 3.2.6
Disconnect and Reconnect Handling	3.2.6
External Interface Requirements	3.1
Leaderboard	3.2.8
Logical Database Requirements	3.4
Maintainability	3.6.4
Match	1.4; 3.2.4
Match History	3.2.7
Matchmaking Queue	1.4; 3.2.2
Non-Functional Requirements (Software System Attributes)	3.6
Operating Environment	2.4
Performance Requirements	3.3
Player Identification	3.2.1
Portability	3.6.7
Product Functions	2.2
Product Perspective	2.1
Real-Time Combat	3.2.4
Reliability	3.6.1
Robustness	3.6.2
Scope	1.3
Security	3.6.6
Server / Game Server	1.4; 2.1; 3.6.1
Tick	1.4; 3.2.4; 3.3
Usability	3.6.5
User Characteristics	2.3
WebSocket Contract	1.4; 3.1.4; Appendix A
Win Condition	1.4; 3.2.5